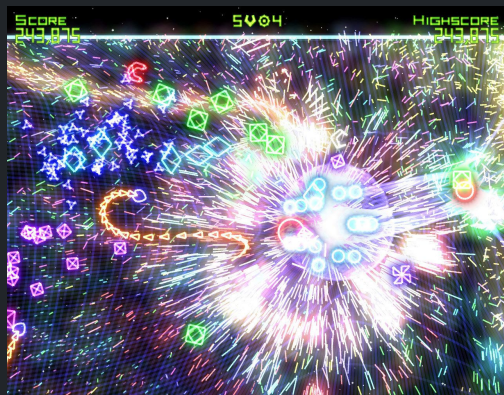


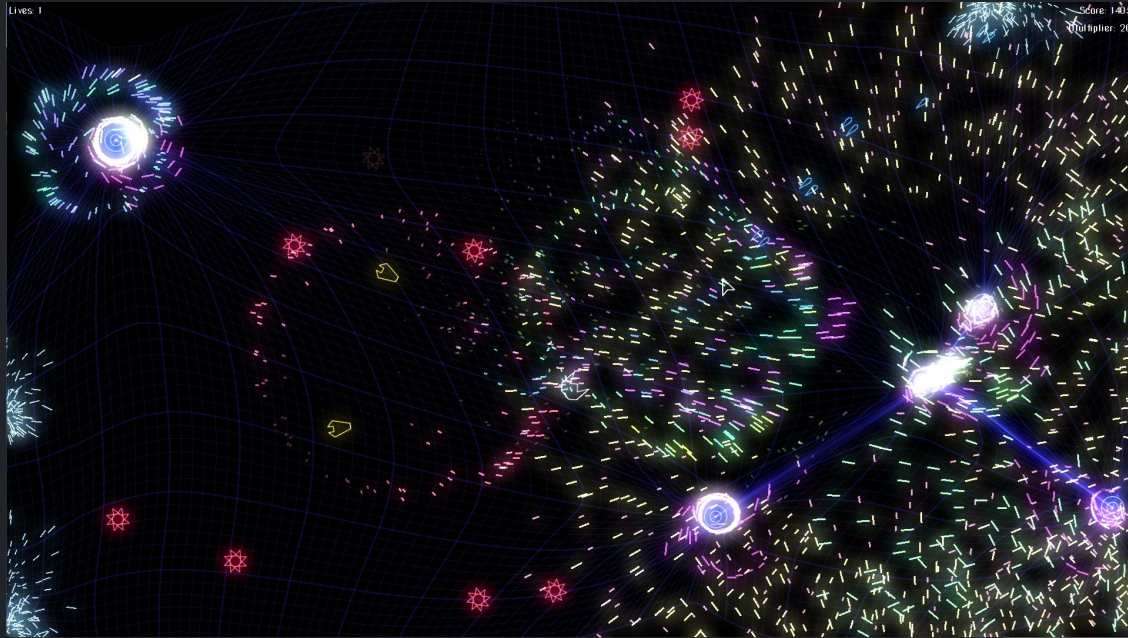
GMD2

Lecture 10: Geometry Wars

Geometry Wars



Today's Goal



Today's Topics

- Component Pattern
- Data Oriented Design
- Object Pool Pattern
- Flyweight Pattern
- Shaders

Why Geometry Wars?

- Thousands of entities on screen at 60 FPS
- Naive OOP collapses: deep inheritance, GC churn, cache misses
- Forces real architectural decisions
 - Every pattern today fixes a real bottleneck
- The iconic neon glow and grid warp are pure shader work

Source Code

- All Source code for today's game can be found here:
 - <https://github.com/Metamate/gmd2-geometrywars>
- Walkthrough in [README.md](#)
- The goal is not to understand all the systems, but the overall architecture employed

Explore the Codebase

Take ~10 minutes to browse the code. Use the README as a guide if you need it.

- Clone the repository and build and play the game
 - How high a score can you get? ;)
- How is the codebase split between the core library and the geometry wars-specific project?

High-Level Architecture

- Game1 owns the application shell
- PlayState owns one active gameplay state
- PlaySession owns one run of the game
- EntityWorld updates entities and collisions
- EntityFactory builds entities by composing components
- Components implement the actual behavior
- Systems handle some cross-component concerns

Game1 → PlayState → PlaySession → EntityWorld / EntityFactory → Components

Component Pattern - The Problem

- Inheritance seems natural: Enemy → ShootingEnemy → HomingShootingEnemy...
- But every new behavior combination doubles the hierarchy
- Want a homing enemy that also explodes? Copy-paste the homing code
- The diamond problem, fragile base classes, and dead code everywhere
- Behavior should be composed, not inherited

Component Pattern - The Solution

- An Entity is just an ID plus a bag of components
- Components hold data and a small slice of behavior
- Transform, Sprite, Collider, Health, AIController, Weapon...
- Mix and match: a HomingExplodingEnemy is just Homing + Explode components
- Systems iterate over entities that have the components they care about
- Favor composition over inheritance
 - The classic OOP maxim, made concrete
- Components are reusable
 - If you keep the “Single Responsibility Principle” in mind

Component Pattern - Simple Example

```
public class Entity {  
    public int Id;  
    public List<Component> Components = [];  
}  
  
public abstract class Component { public Entity Owner; }  
  
public class Transform : Component { public Vector2 Position; public float Rotation, Scale; }  
public class Sprite      : Component { public Texture2D Texture; public Color Tint; }  
public class Health      : Component { public int HP; }  
  
// Spawn a grunt enemy: no new class needed  
var grunt = new Entity();  
grunt.Components.Add(new Transform { Position = spawnPoint });  
grunt.Components.Add(new Sprite    { Texture = gruntTex });  
grunt.Components.Add(new Health    { HP = 1 });
```

Component Pattern Implementation

- Component.cs
- Entity.cs
- EntityFactory.cs

Data Oriented Design

- Think in Data, Not Objects
- OOP asks: "What is a bullet?" — DOD asks: "How does bullet data flow?"
- Array of Structs (AoS)
 - `Bullet[]` — each bullet is one object with all its fields
- Struct of Arrays (SoA)
 - Separate `float[]` `positionsX`, `positionsY`, `velocitiesX`, ...
- SoA packs the fields you actually touch tightly together in memory
- Update loop reads only what it needs
 - Nothing else is dragged into cache

Memory Layout Matters

- L1 cache hit: ~ 1 ns
 - Main memory miss: ~ 100 ns — a 100 $\times$ penalty
- A CPU fetches 64 bytes at a time (one cache line)
 - Make them count
- AoS Bullet { Pos, Vel, Sprite, Owner, Tag, ... } ≈ 80 bytes per object
- Updating positions drags sprites, tags, and pointers through cache too
- SoA positions[]: 16 bullets fit in one cache line
 - 16 $\times$ the throughput
- Thousands of bullets flying? This is the difference between 60 FPS and 15

SoA Example

```
public class BulletSystem {  
    const int MAX = 10_000;  
  
    public Vector2[] Position = new Vector2[MAX];  
    public Vector2[] Velocity = new Vector2[MAX];  
    public float[] Lifetime = new float[MAX];  
    public bool[] Active = new bool[MAX];  
    int count;  
  
    public void Update(float dt) {  
        for (int i = 0; i < count; i++) {  
            if (!Active[i]) continue;  
            Position[i] += Velocity[i] * dt;  
            Lifetime[i] -= dt;  
            if (Lifetime[i] <= 0) Active[i] = false;  
        }  
    }  
}
```

SoA Implementations

- Grid.cs
- ParticleManager.cs

Object Pool - The Problem

- You fire 20 bullets per second.
 - Each hit spawns 30 particles.
 - Each enemy dies into 50 debris pieces
- That's thousands of new allocations per second
 - All short-lived
- C# is a managed language: **new** means the GC will eventually clean up
- GC pauses are non-deterministic
- A stutter every few seconds is the telltale sign of allocation-in-the-hot-loop

Object Pool - Simple Example

```
public class ParticlePool {  
    Particle[] items = new Particle[5000];  
    public ParticlePool() {  
        for (int i = 0; i < items.Length; i++) items[i] = new Particle();  
    }  
  
    public Particle Spawn(Vector2 pos) {  
        for (int i = 0; i < items.Length; i++) {  
            if (!items[i].Active) {  
                items[i].Reset(pos);  
                items[i].Active = true;  
                return items[i];  
            }  
        }  
        return null; // pool exhausted - size it generously  
    }  
}
```

Object Pool Implementation

- ObjectPool.cs
- BulletSpawner.cs

Flyweight Pattern

- Share What's Shared, Store What's Unique
- 500 enemies on screen
 - But they're all the same sprite
- Intrinsic state: shared, immutable
 - The Texture2D, the mesh, the stats template
- Extrinsic state: per-instance
 - Position, velocity, current HP, color tint
- One EnemyType object holds intrinsic data
 - Each Entity references it
- 500 entities × 4-byte pointer to one texture, not 500 copies of the texture
- Memory: kilobytes instead of megabytes. Load time: instant instead of slow

Flyweight Example

```
// Intrinsic (shared) – one instance, referenced everywhere
public class EnemyType {
    public Texture2D Texture;
    public float MaxSpeed, SpawnHP;
    public int    ScoreValue;
}

// Extrinsic (per-instance) – one per live enemy
public struct EnemyInstance {
    public EnemyType Type;    // pointer – the flyweight
    public Vector2    Position, Velocity;
    public int        CurrentHP;
}

// Draw: SpriteBatch batches draws of the same texture automatically
foreach (var e in enemies)
    spriteBatch.Draw(e.Type.Texture, e.Position, Color.White);
```

Flyweight Implementations

- GameAssets.cs
- GameplayDefinitions.cs

Shaders

- Moving work to the GPU
- CPU has ~8 cores. GPU has thousands
 - But they all run the same program
- Vertex shader: runs once per vertex
 - Transforms positions, can displace geometry
- Pixel (fragment) shader: runs once per pixel.
 - Decides the final color
- MonoGame ships with a default shader
 - You can write custom ones in HLSL (.fx files)
- Any per-pixel effect
 - Glow, blur, distortion, color grading
- The CPU submits draw calls; the GPU paints pixels in parallel

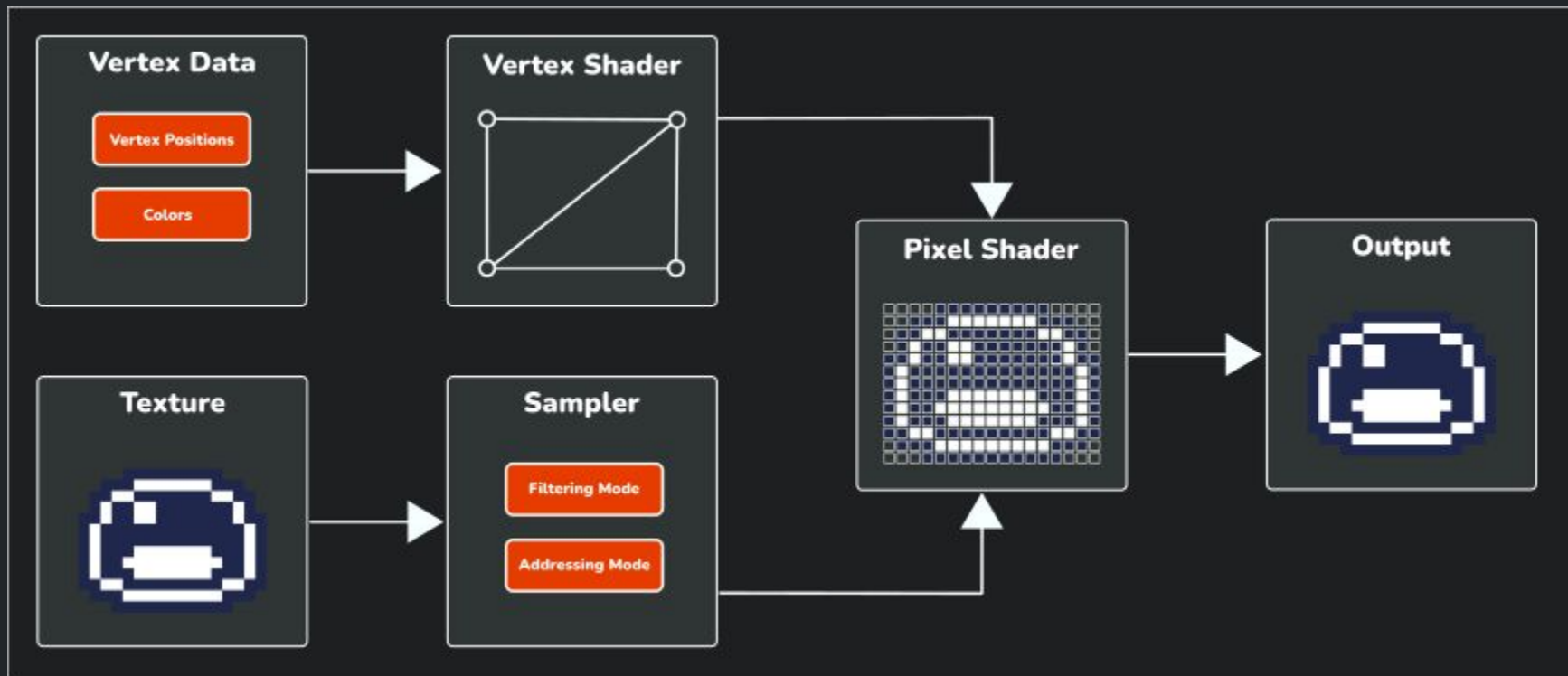
Bloom, The Signature Geometry Wars Glow

Bloom makes bright pixels bleed light into their neighbors

1. Render the scene to a `RenderTarget2D` instead of the back buffer
2. Threshold pass — keep only pixels brighter than some cutoff
3. Gaussian blur the bright pixels — horizontal pass, then vertical
4. Additively blend the blurred result back over the original scene

Each pass is a fullscreen quad with a different pixel shader (post-processing)

The Shader Pipeline



Summary

- Structure
 - Component Pattern defines what an entity is
- Memory Layout
 - Data-Oriented Design defines how entities live in memory
- Memory Allocation
 - Object Pool decides when entities are allocated
- Shared Memory (deduplication)
 - Flyweight decides what entities share
- Presentation
 - Shaders decide how entities look